Práctica 2: Clasificación de Transacciones mediante PLN

Autores:

Diego Esclarín (51729611N)
Sofía Contreras (09848471D)

Aplicaciones de Inteligencia Artificial
Grado en Ingeniería en Inteligencia Artificial
Universidad Rey Juan Carlos
Curso 2025–26

Índice

1. Introducción **3**

2. Contexto **5**

3. Estado del arte **7**

4. Propuesta de solución **9**

4.1. Dataset Unificado y Preprocesamiento 9
4.2. Comparativa de Modelos 9
4.3. Resultados y Selección Final 10
4.4. Comparativa con Modelos Transformer 11

5. Conclusiones **13**

1. Introducción

El propósito fundamental de este proyecto es el desarrollo de una solución inteligente basada en Procesamiento de Lenguaje Natural (PLN) para automatizar uno de los procesos más tediosos en la gestión financiera: la clasificación de movimientos bancarios. En un entorno digital donde el volumen de transacciones personales crece exponencialmente, la capacidad de organizar automáticamente estos datos deja de ser un lujo para convertirse en una necesidad técnica. El sistema diseñado actúa como un puente entre la información desestructurada que emiten las entidades bancarias y la estructura organizada que requiere un usuario para comprender sus hábitos de gasto.

El núcleo de la investigación se centra en la capacidad de interpretación semántica. A diferencia de los sistemas tradicionales basados en reglas rígidas o búsqueda de palabras clave —que suelen fallar ante variaciones ortográficas o sinónimos—, este sistema se enfoca en comprender la intención detrás de cada descripción. Para lograr esta robustez, el proceso de entrenamiento utiliza un corpus de datos diseñado para cubrir un amplio espectro de posibilidades lingüísticas. Se busca que el modelo no solo reconozca términos aislados, sino que sea capaz de gestionar la variabilidad con la que una misma operación financiera puede ser registrada, garantizando así una categorización coherente y precisa en escenarios de uso real.

Finalmente, el proyecto destaca por su capacidad de mejora personalizada. El sistema no es una herramienta estática, sino que permite integrar las correcciones y las nuevas descripciones que el usuario genere en su día a día. Al reentrenar el modelo con estos datos específicos, el clasificador deja de ser una solución generalista para adaptarse al vocabulario y a los hábitos de gasto particulares de cada persona. De esta forma, el sistema garantiza una precisión creciente, optimizando su rendimiento a medida que se acumula un historial de transacciones real y verificado por el propio usuario.

2. Contexto

El problema abordado es la ineficiencia de indicar manualmente la categoría de gasto de una transacción bancaria. Además de muchas de estas transacciones se repiten recurrentemente, lo que hace que sea una tarea tediosa para el usuario y pueda generar bajas en el uso de la aplicación.

La heterogeneidad en las descripciones de las transacciones, sumada al volumen de movimientos, impide el uso de reglas fijas o filtros de texto simples, que fallan ante cualquier cambio en la redacción. Por ello, es necesario un sistema basado en Procesamiento de Lenguaje Natural (PLN) más complejo que recoja algo más de información y correlacione para clasificarlas automáticamente en categorías específicas (*Area*).

Para validar la eficacia de esta clasificación, se utilizan las siguientes métricas:

- **Accuracy (Exactitud):** Mide el porcentaje total de transacciones que el modelo ha clasificado en la categoría correcta.
- **F1-Score:** Evalúa el equilibrio entre la precisión y la capacidad de recuperación del modelo. Es fundamental para asegurar que el sistema clasifique bien tanto las categorías muy comunes (como *Food*) como aquellas con menos ejemplos en el dataset.

El objetivo es transformar este flujo de descripciones heterogéneas en información organizada y etiquetada de forma automática.

3. Estado del arte

La clasificación de textos financieros ha evolucionado desde sistemas basados en reglas hacia modelos de aprendizaje automático capaces de gestionar la ambigüedad del lenguaje natural.

- **Técnicas de vectorización:** Para transformar texto en datos numéricos, destaca el uso de *TF-IDF*, que asigna pesos a los términos según su relevancia en el corpus, y los *Word Embeddings*, que permiten capturar relaciones semánticas complejas al representar palabras en espacios vectoriales continuos [1].

- **Modelos de clasificación:** El uso de algoritmos clásicos como *SVM* y *Random Forest* sigue siendo fundamental en entornos que requieren eficiencia computacional y robustez en textos cortos. Últimos estudios de 2025 confirman que estos modelos ofrecen una mayor consistencia en la clasificación de etiquetas específicas frente a arquitecturas más pesadas [2]. Aunque los *Transformers* marcan el estado del arte en precisión, su alta demanda de recursos plantea desafíos de escalabilidad en aplicaciones ligeras [1].

- **Aprendizaje incremental (Online Learning):** En el sector financiero, es crucial que los modelos se adapten a nuevos patrones de gasto sin reentrenamientos completos. La automatización de estos procesos requiere algoritmos optimizados (como SGD) que permitan un aprendizaje continuo y eficiente para garantizar la escalabilidad en sistemas de producción [3], [4].

4. Propuesta de solución

Para abordar la clasificación de transacciones financieras, se ha desarrollado un pipeline modular que abarca desde el preprocesamiento de texto hasta la evaluación de múltiples modelos de aprendizaje supervisado.

4.1. Dataset Unificado y Preprocesamiento

Después de hacer tests con el dataset original, nos dimos cuenta de que no era suficiente para entrenar un modelo de machine learning, por lo que decidimos combinarlo con el dataset modificado. Aun así, al ser datasets tan diferentes, no evaluaban corréctamente el resto de categorías, por lo que decidimos quedarnos solo con el dataset modificado.

Se han combinado dos fuentes de datos principales: `db_orig.csv` y `db_modified_descriptions.csv`, creando un corpus robusto con el doble de ejemplos por categoría y que ha funcionado mejor. Pero la diferencia era demasiado grande y aunque ajustaba mejor decidimos supervisar y depurar la creación de descripciones creando la base de datos `db_mod_descript.csv` que ha mejorado aún más las predicciones. Recordando la importancia que se habló en la primera práctica sobre la gran importancia de la creación y depuración de datos en los problemas que estamos afrontando.
El preprocesamiento incluye:

- Conversión a minúsculas y eliminación de caracteres especiales.
- Vectorización de texto mediante **TF-IDF** (*Term Frequency - Inverse Document Frequency*) para transformar las descripciones en representaciones numéricas.

4.2. Comparativa de Modelos

Se evaluaron cuatro arquitecturas distintas mediante una validación con partición 50/50 para asegurar la capacidad de generalización:

- **Logistic Regression**: Proporcionó alta explicabilidad pero rendimiento ligeramente inferior en clases minoritarias.
- **SGDClassifier**: El modelo base del sistema, destaca por su extrema eficiencia en recursos.
- **Random Forest**: Elevada precisión, pero con un coste computacional y de almacenamiento excesivo para el dominio del problema.
- **Linear SVM**: El modelo con mejor rendimiento global.

4.3. Resultados y Selección Final

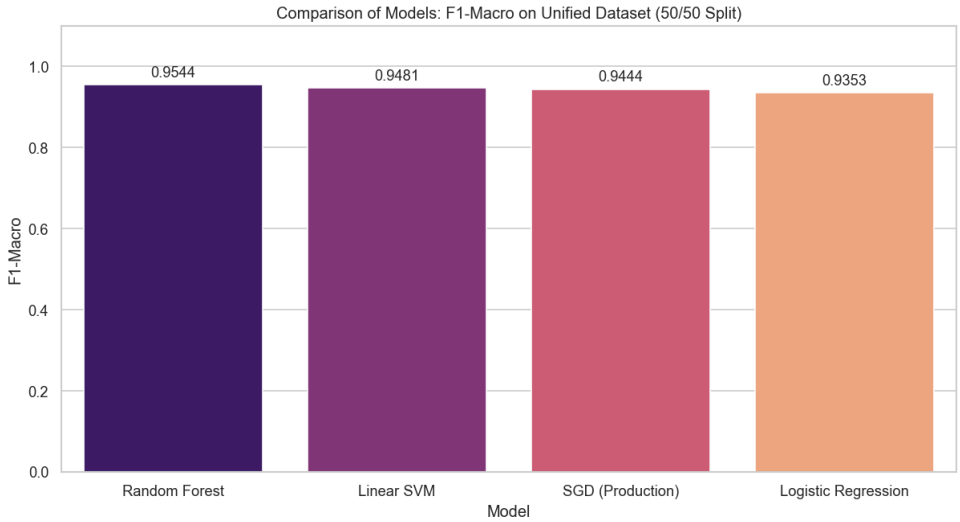

Figura 1: Resultados de la comparativa con el dataset conjunto

Tras las pruebas, el modelo **Linear SVM** obtuvo un **F1-Macro de 0.9481** con el dataset unificado 1 y **F1-Macro de 1** con el dataset modificado y revisado, superando al SGD original. Este modelo ofrece el mejor equilibrio entre:

1. **Precisión**: Máxima eficacia en la separación de categorías complejas (como "Ocio" vs "Ocio, Vacaciones").

2. **Explicabilidad**: Al ser un modelo lineal, permite auditar los pesos de las palabras clave.

3. **Eficiencia**: Mantiene una huella de memoria ligera y tiempos de inferencia casi instantáneos.

Por tanto, se ha seleccionado `LinearSVM.joblib` como el modelo principal para la implementación en el sistema de producción.

4.4. Comparativa con Modelos Transformer

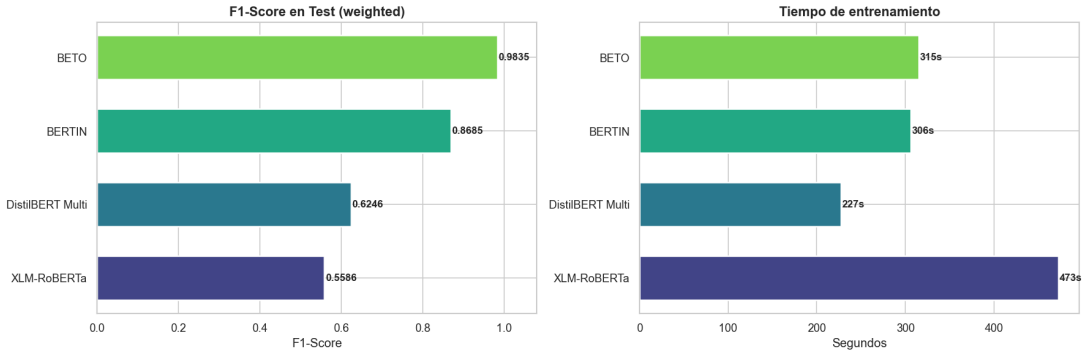

Figura 2: Comparativa de modelos Transformer: F1-Score y tiempo de entrenamiento

Además de los modelos clásicos, se realizaron pruebas con modelos **Transformer** preentrenados, aplicando *fine-tuning* sobre el mismo dataset para clasificar el área a partir de la descripción. Se evaluaron cinco modelos: tres monolingües en español (**BETO**, **RoBERTa-BNE** y **BERTIN**) y dos multilingües (**XLM-RoBERTa** y **DistilBERT Multilingual**).

Como se observa en la Figura 2, el mejor modelo Transformer fue **BETO** con un F1-Score de 0.9835, seguido de BERTIN (0.8685). Los modelos multilingües obtuvieron resultados significativamente inferiores: DistilBERT Multi (0.6246) y XLM-RoBERTa (0.5586).

Sin embargo, incluso el mejor Transformer no supera al **Linear SVM** (F1 = 1.0), y el coste computacional es drásticamente mayor: cada modelo Transformer requiere entre 227 y 473 segundos de entrenamiento en CPU, frente a tiempos casi instantáneos para los modelos lineales. Esto se debe a que las descripciones del dataset son textos cortos con vocabulario muy predictivo, donde TF-IDF captura perfectamente las señales discriminativas sin necesidad de representaciones contextuales profundas.

Por estas razones, para textos con estas características y un único usuario, nos decantamos por los modelos lineales clásicos, que ofrecen mayor precisión con un coste computacional mínimo.

5. Conclusiones

El desarrollo de este sistema demuestra que modelos lineales clásicos como el **Linear SVM**, combinados con una vectorización TF-IDF bien configurada, son extremadamente competitivos para tareas de clasificación de texto financiero corto.

La unión de los datasets original y modificado ha sido clave para mejorar la robustez ante diferentes estilos de redacción. La adopción final del modelo SVM garantiza que el sistema no solo sea preciso, sino también transparente y eficiente para su despliegue real.

La comparativa con modelos Transformer ha confirmado que, aunque estos modelos representan el estado del arte en PLN general y funcionan mejor con datos heterogéneos de múltiples usuarios, para un dataset pequeño con descripciones cortas y vocabulario predictivo de un único usuario, los modelos lineales clásicos son la opción más adecuada.

Como líneas futuras, se proponen dos vías de mejora:

- **Ajuste fino de hiperparámetros** (*Hyperparameter Tuning*) para optimizar la precisión en las categorías combinadas.
- **Estrategia híbrida para nuevos usuarios**: utilizar un modelo Transformer preentrenado (como BETO) como clasificador inicial para usuarios nuevos que aún no disponen de datos propios, ya que generaliza bien sin necesitar ejemplos específicos. Una vez que el usuario acumule una base de datos consistente, migrar automáticamente a un modelo **Linear SVM + TF-IDF**, mucho más eficiente y preciso para su vocabulario particular.
- **Mejora de la vectorización**: explorar técnicas de vectorización más avanzadas, como Word2Vec o FastText, para capturar mejor el significado semántico de las palabras.
- **Internacionalización**: Generación de corpus sintéticos en otros idiomas para internacionalizar la base de datos.

Referencias Bibliográficas

[1] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu y J. Gao, «Deep Learning–based Text Classification: A Comprehensive Review,» *ACM Computing Surveys (CSUR)*, 2021.

[2] J. S. John, B. R. Prathap, G. Gupta y J. Melanaturu, «Categorizing Mental Stress: A Consistency-Focused Benchmarking of ML and DL Models via Taxonomy-Driven NLP Techniques,» *Natural Language Processing Journal*, 2025.

[3] A. T. Oyewole, O. B. Adeoye, W. A. Addy, C. C. Okoye, O. C. Ofodile y C. E. Ugochukwu, «Automating financial reporting with natural language processing: A review and case analysis,» *World Journal of Advanced Research and Reviews*, 2024.

[4] J. Kolluri, S. Razia y S. R. Nayak, «Text classification using Machine Learning and Deep Learning Models,» *International Journal of Recent Technology and Engineering (IJRTE)*, 2019.